Csmall: especificação da linguagem

Compiladores

1 Objetivo

Desenvolver um compilador simples para a linguagem Csmall. A implementação deve:

- **Lexer:** reconhecer os tokens definidos nesta especificação.
- **Parser:** implementar a gramática por descida recursiva e verificar se o programa respeita a sintaxe da linguagem.
- **Tabela de símbolos:** armazenar o nome, o tipo e o escopo das variáveis.
- **Análise semântica:** aplicar as regras semânticas desta especificação, verificando declaração antes do uso, redeclaração no mesmo escopo e compatibilidade de tipos.

O compilador deve identificar erros léxicos, sintáticos e semânticos e apresentar uma mensagem que indique a causa e, quando possível, a linha do programa.

2 Características da linguagem

Csmall possui os tipos `int` e `float`; declarações de variáveis; atribuições; expressões aritméticas; comparações numéricas; `print`; condicional `if`, com `else` opcional; repetição `while`; e blocos delimitados por chaves. Os corpos de `if` e `while` devem ser blocos entre chaves. As declarações devem aparecer no início do programa e no início de cada bloco, antes dos comandos desse escopo.

Não há funções, vetores, strings, ponteiros, operadores lógicos ou conversões implícitas de tipos. O operador de subtração é binário; números negativos unários não fazem parte desta versão.

3 Tokens

3.1 Espaços, comentários, identificadores e literais

Espaços, tabulações e quebras de linha não formam tokens. Comentários começam com `//` e terminam na quebra de linha. O lexer deve registrar a categoria, o lexema e a linha de origem de cada token. Um caractere que não pertença a nenhum token válido e não faça parte de um espaço ou comentário deve gerar erro léxico.

Token	Regra / exemplo
ID	`[A-Za-z_][A-Za-z0-9_]*`
INT_CONST	`[0-9]+`
FLOAT_CONST	`[0-9]+\.[0-9]+`

3.2 Palavras reservadas

int	float	if	else	while	print

Uma sequência que satisfaz a regra de identificador é inicialmente reconhecida como identificador; depois, o lexema é comparado com as palavras reservadas.

3.3 Operadores e delimitadores

Símbolo	Token	Símbolo	Token
`+`	PLUS	`-`	MINUS
`*`	MULT	`/`	DIV
`=`	ASSIGN	`<`	LT
`<=`	LE	`>`	GT
`>=`	GE	`==`	EQ
`!=`	NE	`(`	LPAREN
`)`	RPAREN	`{`	LBRACE
`}`	RBRACE	`;`	SEMICOLON

Os operadores de dois caracteres devem ser reconhecidos usando correspondência máxima, para que, por exemplo, `<=` seja um único token `LE`, e não os tokens `LT` e `ASSIGN`.

3.4 Lista completa de tokens

Token	Lexemas / significado
`INT, FLOAT`	`int`, `float`
`IF, ELSE, WHILE, PRINT`	`if`, `else`, `while`, `print`
`ID`	identificador
`INT_CONST, FLOAT_CONST`	constantes numéricas
`PLUS, MINUS, MULT, DIV`	`+`, `-`, `*`, `/`
`ASSIGN`	`=`
`LT, LE, GT, GE, EQ, NE`	`<`, `<=`, `>`, `>=`, `==`, `!=`
`LPAREN, RPAREN`	`(`, `)`
`LBRACE, RBRACE`	`{`, `}`
`SEMICOLON`	`;`
`EOF`	fim da entrada

4 Gramática

A gramática abaixo define a sintaxe de Csmall. Os símbolos terminais aparecem em negrito; ε representa a cadeia vazia.

$$
\begin{aligned}
\mathit{Programa} &\rightarrow \mathit{ListaDecl}\ \mathit{ListaCmd} \\
\mathit{ListaDecl} &\rightarrow \mathit{Declaracao}\ \mathit{ListaDecl} \mid \varepsilon \\
\mathit{Declaracao} &\rightarrow \mathit{Tipo}\ \texttt{ID ;} \\
\mathit{Tipo} &\rightarrow \texttt{int} \mid \texttt{float} \\
\mathit{ListaCmd} &\rightarrow \mathit{Comando}\ \mathit{ListaCmd} \mid \varepsilon \\
\mathit{Comando} &\rightarrow \mathit{Atribuicao} \mid \mathit{Print} \mid \mathit{If} \mid \mathit{While} \mid \mathit{Bloco} \\
\mathit{Atribuicao} &\rightarrow \texttt{ID =}\ \mathit{Expressao}\ \texttt{;} \\
\mathit{Print} &\rightarrow \texttt{print (}\ \mathit{Expressao}\ \texttt{) ;} \\
\mathit{If} &\rightarrow \texttt{if (}\ \mathit{Condicao}\ \texttt{)}\ \mathit{Bloco}\ \mathit{OpcionalElse} \\
\mathit{OpcionalElse} &\rightarrow \texttt{else}\ \mathit{Bloco} \mid \varepsilon \\
\mathit{While} &\rightarrow \texttt{while (}\ \mathit{Condicao}\ \texttt{)}\ \mathit{Bloco} \\
\mathit{Bloco} &\rightarrow \texttt{\{}\ \mathit{ListaDecl}\ \mathit{ListaCmd}\ \texttt{\}} \\
\mathit{Condicao} &\rightarrow \mathit{Expressao}\ \mathit{RelOp}\ \mathit{Expressao} \\
\mathit{RelOp} &\rightarrow \texttt{<} \mid \texttt{<=} \mid \texttt{>} \mid \texttt{>=} \mid \texttt{==} \mid \texttt{!=} \\
\mathit{Expressao} &\rightarrow \mathit{Termo}\ \mathit{ExprLinha} \\
\mathit{ExprLinha} &\rightarrow \texttt{+}\ \mathit{Termo}\ \mathit{ExprLinha} \mid \texttt{-}\ \mathit{Termo}\ \mathit{ExprLinha} \mid \varepsilon \\
\mathit{Termo} &\rightarrow \mathit{Fator}\ \mathit{TermoLinha} \\
\mathit{TermoLinha} &\rightarrow \texttt{*}\ \mathit{Fator}\ \mathit{TermoLinha} \mid \texttt{/}\ \mathit{Fator}\ \mathit{TermoLinha} \mid \varepsilon \\
\mathit{Fator} &\rightarrow \texttt{ID} \mid \texttt{INT_CONST} \mid \texttt{FLOAT_CONST} \mid \texttt{(}\ \mathit{Expressao}\ \texttt{)}
\end{aligned}
$$

5 Regras semânticas que devem ser aplicadas

O `else` é opcional. Como os corpos de `if` e `while` devem ser blocos entre chaves, a gramática evita a ambiguidade de associar um `else` a condicionais aninhadas. Cada bloco pode conter suas próprias declarações, antes dos comandos.

1. Uma variável deve ser declarada antes de ser usada.
2. Uma variável não pode ser declarada duas vezes no mesmo escopo.
3. O programa possui um escopo global. Cada bloco `{...}` cria um novo escopo.
4. Uma variável declarada em escopo interno pode ocultar uma variável de escopo externo. A busca começa no escopo atual e segue para os escopos externos.
5. Uma constante `INT_CONST` tem tipo `int`; uma constante `FLOAT_CONST` tem tipo `float`; o tipo de um identificador é obtido na tabela de símbolos.
6. Os operadores aritméticos recebem operandos numéricos. O resultado é `int` quando ambos os operandos são `int`; caso contrário, é `float`.
7. O tipo da expressão atribuída deve ser igual ao tipo da variável de destino. Não há conversão implícita entre `int` e `float`.
8. A condição de `if` e `while` deve comparar duas expressões numéricas. Os lados podem ter tipos numéricos diferentes.
9. `print` recebe uma expressão numérica.

Ao entrar em um bloco, a tabela de símbolos deve abrir um novo escopo; ao sair dele, deve encerrar esse escopo. A busca por identificadores começa no escopo atual e continua pelos escopos externos. A análise semântica deve consultar a tabela para obter tipos e escopos e informar um erro quando alguma dessas regras for violada.

6 Exemplos

Programa válido:

```
int x;
float y;
x = 2 + 3 * 4;
y = x + 1.5;
print(y);
if (x < 5) {
    print(x);
}
if (x != 10) {
    print(x);
} else {
    print(y);
}
while (x < 10) {
    x = x + 1;
}
```

Exemplo de escopos e variável ocultada:

```
int valor;
valor = 10;
{
    float valor;
    valor = 2.5;
    print(valor);
}
print(valor);
```

Exemplos inválidos (cada trecho representa um programa separado):

```
// 1. Variavel nao declarada (execute isoladamente)
x = 10;
// 2. Redeclaracao no mesmo escopo (execute isoladamente)
int x; int x;
// 3. Tipo incompativel (execute isoladamente)
float y; y = 3;
// 4. Condicao sem operador relacional (execute isoladamente)
int y; if (y) { print(y); }
```